

Anatomy of a Regular Expression

One pattern, every part labeled. Read it left to right, the way the engine does.

`/^[A-Z]{2}-\d{3,4}$/i`

`^`

Anchor: must match from the very START of the string.

`[A-Z]`

Character class: any ONE character from the set.

`{2}`

Quantifier: exactly two of the thing before it.

`-`

A literal hyphen. Most characters just mean themselves.

`\d`

Shorthand for `[0-9]`. Also `\w` word and `\s` space; `\D \W \S` negate.

`{3,4}`

Between three and four. Also `*` zero or more, `+` one or more, `?` optional.

`$`

Anchor: must reach the END. `^` and `$` together mean the WHOLE string must match.

`i`

A flag, after the closing slash. `i` ignore case, `g` find all, `m` multiline.

Matches: `AB-123` · `ab-4567` · `ZZ-999`

Does not: `A-123` · `AB-12` · `AB-12345` · `xAB-123`

Use a regex to check the SHAPE of a short string. Do not use one to parse HTML, and do not invent your own for email; `type="email"` already does it. A pattern with a `g` flag keeps `lastIndex` between calls, so reusing one across a loop can skip matches; make a fresh regex, or use `String.matchAll`.